



Note: To use `getCurrentAcceleration()`, there is no need of the `startWatch()` and `stopWatch()` methods. Just call `navigator.accelerometer.getCurrentAcceleration (onSuccess, onError)` in `deviceReady`.

Geolocation: This provides the current information on the location of the device, i.e., its latitude and longitude. It can fetch this information from the device's GPS, Wi-Fi, Bluetooth, etc. Some devices already have built-in support, in which case this API need not to be used.

Geolocation is used in map applications, and can help you locate hospitals or restaurants nearby, based on the current location of the device.

The methods for using the geolocation are:

1. `getCurrentPosition()` – This is used to get the current position of the device and returns the 'Position' object as a parameter.
4. `WatchPosition` – Watches for any changes in the device's position.
5. `clearWatch()` – This is used to stop watching the changes in position.

The arguments to pass through these methods are:

1. `geolocationSuccess` – This method will be called when the position has been successfully located.
2. `geolocationError` – This method will be called when the device's position could not be located.
3. `geolocationOptions` – This will provide the timeout in milliseconds. And if there is no update found after the time-out, an error will be thrown.



Note: All these methods can be implemented similar to those implemented in `accelerometer.html`, and to retrieve and display latitude and longitude, use '`position.coords.latitude`' and '`position.coords.longitude`' respectively.

Contacts: This API provides support to access the contacts database of the device; it allows you to create a new contact and find a particular one.

Usage: Applications can see the contact list saved in the device's database.

The methods for using the contacts API are:

1. `contacts.create` – This will create a new contact with some details about the contact.
2. `contacts.save` – This will persist the newly created contact into the contacts database.
3. `contacts.find` – This will find the particular contact based on the search parameter.

Arguments to be passed in `contacts.find()` are:

1. `contactFields` – This method will decide which properties should be returned/displayed for a contact.
2. `ContactSuccess` – This callback function will be returned when a contact has been successfully retrieved from the database.



Figure 5: Creating a contact

```

Contacts.html
// Creating a contact
<!DOCTYPE html><html>
<head> <title>Contact
Example</title>
<script type="text/javascript" charset="utf-8"
src="cordova-2.7.0.js"></script>
<script type="text/javascript" charset="utf-8">

document.addEventListener("deviceready", deviceReady,
false);

function deviceReady() {
    var addContact = navigator.contacts.
create({"displayName": "Anupriya Sharma"}); (1)
    addContact.note = "This is my contact list"; (2)
    addContact.save(onSuccess,onError); (3)
}
function onSuccess()
{
    alert("Saved");
}
function onError()
{
    alert("Failed");
}
</script> </head> <body>
<h1>Adding a new Contact</h1>
</body>/html>

```

Let us now examine what is happening in the above code.

In Line 1, 'addContact' is the variable that will hold the value of the 'Contact' object which is going to be created by using the 'create' method, and arguments of the create method depict the contact name.

In Line 2, you have just added a note with the newly created contact.

In Line 3, the method 'save()' will actually persist the newly created contact in the database, and will be displayed in the contacts of the emulator/device.

The result of the above code is displayed in Figure 5.

Searching contacts

`contacts.find()` – To implement this method, the following changes should be done in `contacts.html`